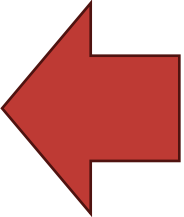
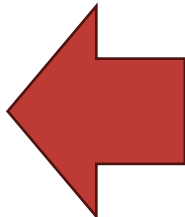
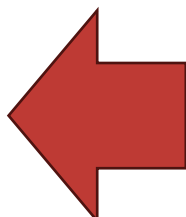
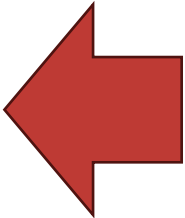
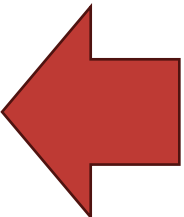
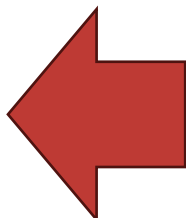


Target Scope (Features)

- ~~1. Agent Version Control / Audit Logs~~
2. Agent Follow up users on Inactive sessions or trigger API ** UI required **
3. *Widget builder tool for widget with LLM or manual* ** UI required **
4. Analytic dashboard revamp for agentic and flexible data points ** UI required **
- ~~5. 2FA Integration (Authenticator Integration) ** UI required **~~
- ~~6. 3rd Party Crawler Integration~~

AI Experimental Features

- ❖ License Control 
- ❖ Agent Version Control / Audit Logs 
- ❖ Agent Follow up users on Inactive sessions or trigger API 
- ❖ Analytic dashboard revamp for agentic and flexible data points 
- ❖ 2FA Integration 
- ❖ 3rd Party Crawler Integration 

Timeline

- Sprint Kick Off – 8 Jan
- Number of Sprints – 3
- Tentative Release – 20 Feb

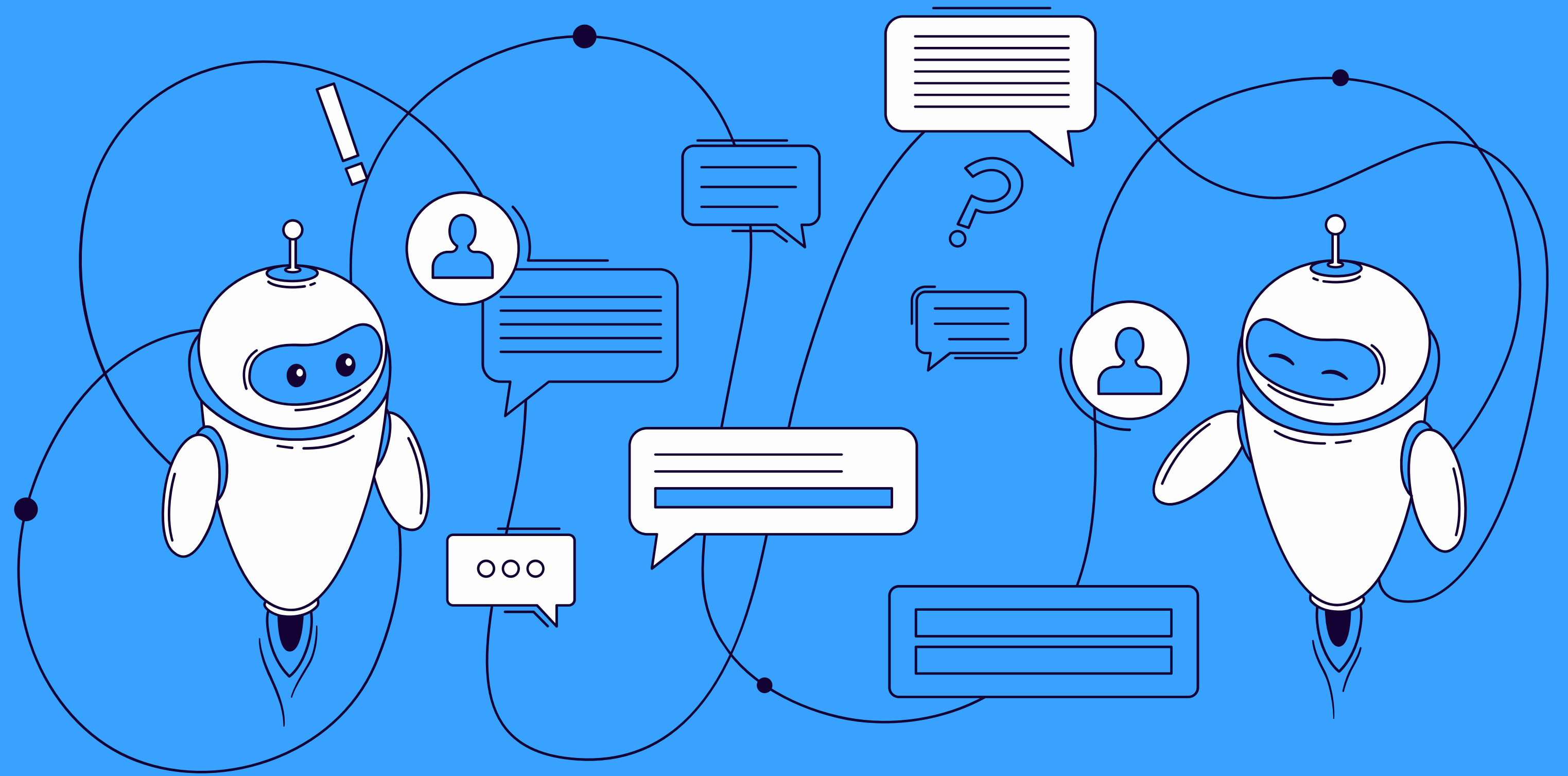
Total Development – 20 days

Bug Fixes & QA – 10 days

Resources

- Temporal integration (or other similar trigger type nodes) – WLP
- Agent as a tool (in progress) – ZLH / HL
- LightRAG – HAK / SLN
- License Control - WLP ←
- A2A (TBC)
- Agent Version Control / Audit Logs – ZLH / HL ←
- Agent Follow up users on Inactive sessions or trigger API – ZLH / HL ←
- Widget builder tool for widget with LLM or manual - ANCM
- Analytic dashboard revamp for agentic and flexible data points - ANCM ←
- 2FA Integration - HAK ←
- 3rd Party Crawler Integration - HLO ←

← Full AI Written Task

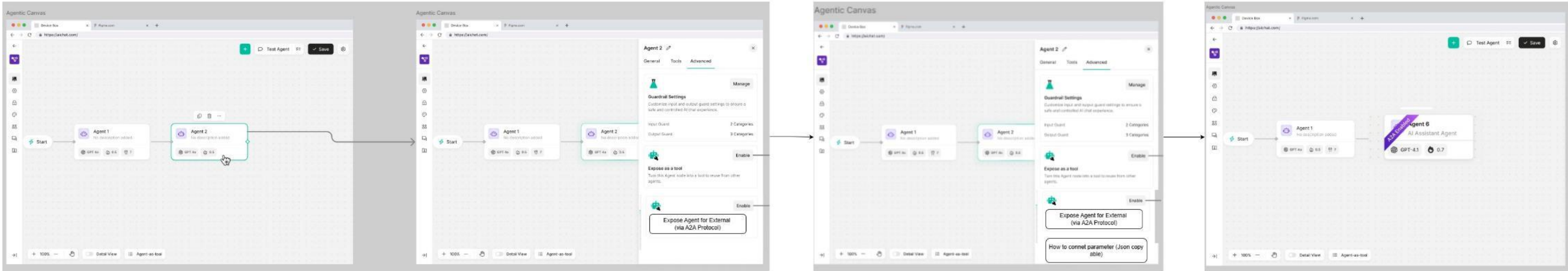


Technical

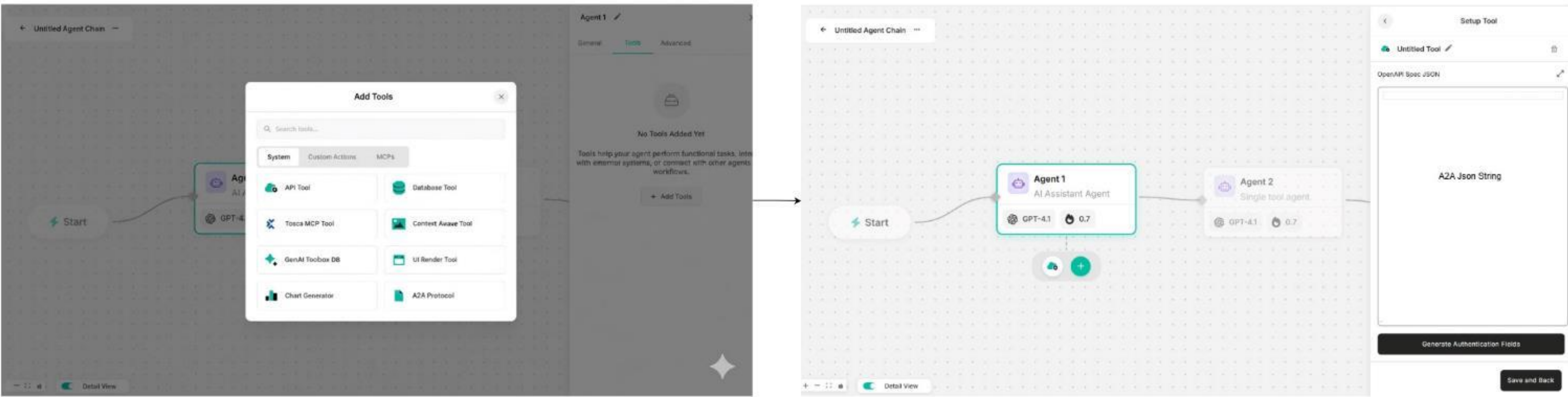
A2A Proposed Flow

Agent to Agent Protocol

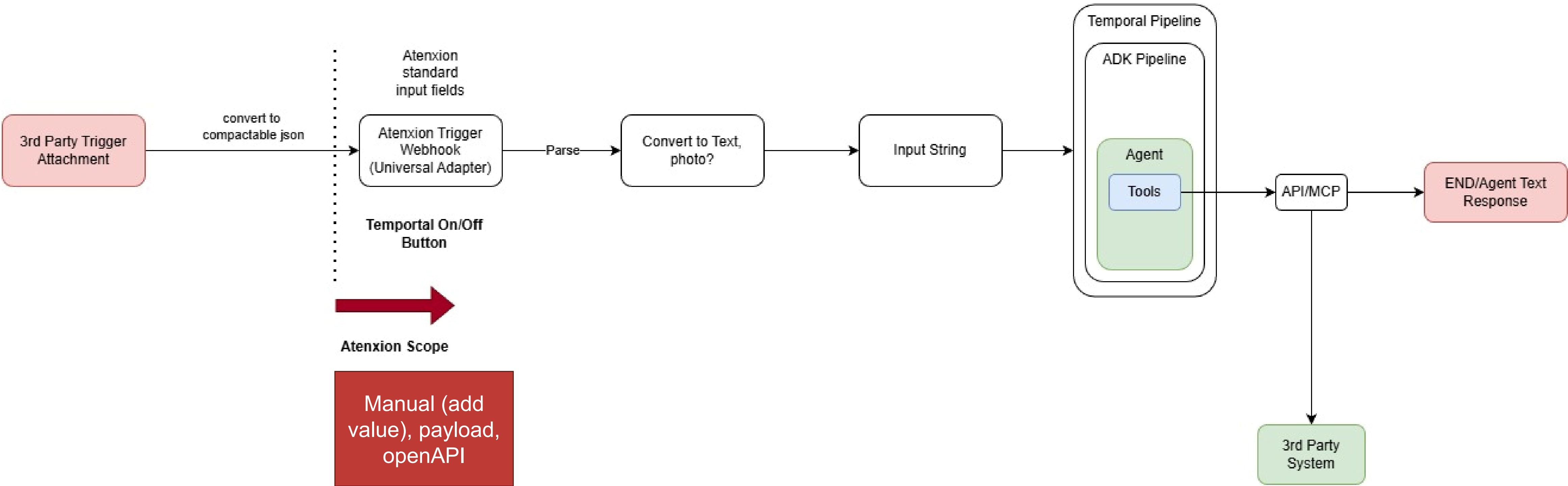
Exposing Agent



Using Agent



Temporal Proposed Flow



Consider adding file reading tools

Trigger Node Wireframe (react)

Atenxion — Trigger Node (Wireframe)
Demo UI for developers (v1)

Agent-first • Minimal triggers

Canvas

Start

Trigger

HR Resume Upload Trigger

On Event (Webhook)

Starts this agent chain when the trigger condition is met.

Webhook

Trigger ID: trg_hr_resume_processing_001

Configure Trigger

Trigger

Configure when this agent chain runs

v1

General

Advanced

Trigger Name

A descriptive name for this trigger.

HR Resume Upload Trigger

Trigger Type

Choose when this agent chain should run.

☒ On Event (Webhook)

Run when an external system sends an event

☐ Manual

Run on-demand from the UI

☐ On Schedule

Run automatically on a schedule

</> Webhook Configuration

Webhook URL

External systems send events to this endpoint.

https://api.atenxion.ai/v1/triggers/events

Copy

Trigger ID

Routes incoming events to this agent chain.

trg_hr_resume_processing_001

Copy

Authentication

How incoming events are verified.

☒ API Key

Simple and effective

☐ HMAC Signature

Signed requests

API Key

Trigger Node Wireframe (react)

Atenxion — Trigger Node (Wireframe)
Demo UI for developers (v1)

Agent-first • Minimal triggers

Canvas

Start

Trigger

HR Resume Upload Trigger

On Event (Webhook)

Starts this agent chain when the trigger condition is met.

Webhook

Trigger ID: trg_hr_resume_processing_001

Configure Trigger

Trigger

Configure when this agent chain runs

v1

General

Advanced

Execution

Execution Mode

Controls whether we start a new workflow per event or signal an existing one.

☒ Per Event

One workflow per event

☐ Per Entity

Signal workflow per entity

Reliability

Deduplication Key

Used to prevent processing duplicate events.

event_id

Ignore events older than

Optionally ignore stale events.

10

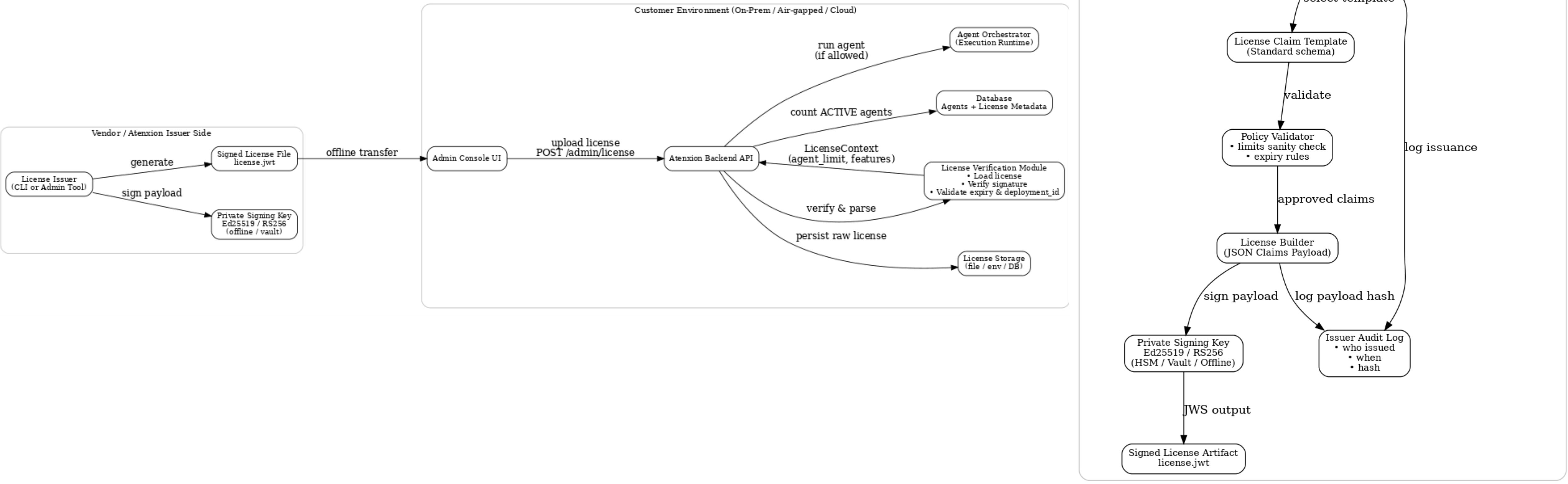
minutes

Tip: triggers should take < 60 seconds to configure.

Cancel

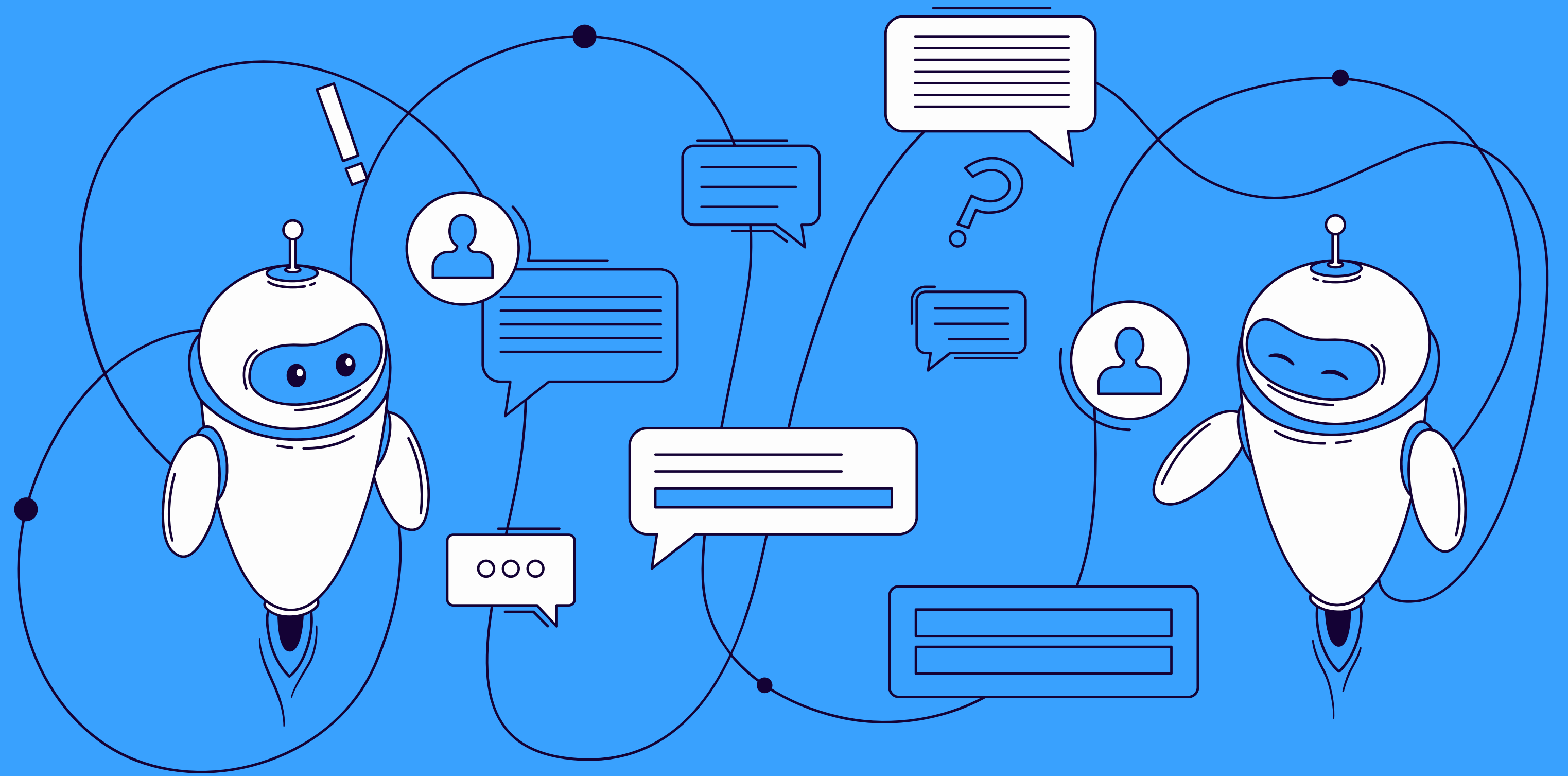
Save

License Proposed Flow



Crawler As Service

Experiment - [Firecrawl - The Web Data API for AI](#)



License Flow

Licensing Features' Goal

- Monetize based on deployed value: multi-user distribution + publishing
- Allow frictionless building/testing: unlimited agents, owner-only test
- Support offline enterprises: no internet dependency (Phase 1)
- Provide clear admin controls: usage visibility + audit logs
- Keep product rules identical across phases to allow migration

Licensing Features' Definitions

- **Agent:** configured assistant/workflow entity with tools/integrations and policies
- **Owner:** creator of agent; always has access
- **Draft:** under construction; not distributed
- **Test Mode:** owner-only execution/validation; integrations allowed; outputs visible only to owner
- **Active (Licensed):** agent eligible for distribution/publishing; consumes 1 license slot
- **Distribution:** any access by users other than owner (team/org/external)
- **Publishing** channels: Embed/widget, WhatsApp, Messenger (extendable)
- **Entitlement:** license limit max_active_agents

Agent Lifecycle & Counting Logic

States

1. **DRAFT**
2. **TEST**
3. **ACTIVE** (licensed)
4. **DISABLED**
5. **ARCHIVED**

License slot counting

- Only **ACTIVE** consumes a slot
- **DRAFT**, **TEST**, **DISABLED**, **ARCHIVED** do not
- Clones always start in **DRAFT**

Core Product Behavior

Allowed in Draft/Test (Unlicensed)

- ✓ Create unlimited agents
- ✓ Configure integrations (data inputs)
- ✓ Run the agent privately as the owner
- ✓ See outputs as owner only

Not allowed until Active (Licensed)

- ✗ Share agent to other users / teams
- ✗ Publish to org-wide catalog
- ✗ Generate embed/widget code
- ✗ Deploy via WhatsApp or Messenger
- ✗ Any multi-user usage (anyone besides owner)

Key principle: licensing controls **access/distribution**, not “ability to integrate.”

Activation / Enforcement

Activation Gatekeeper rule

When owner clicks “Activate”:

If `active_agents_count` $\geq$ `max_active_agents` $\rightarrow$ block activation

Else $\rightarrow$ transition agent to ACTIVE and consume a slot

Deactivation frees license immediately

ACTIVE $\rightarrow$ DISABLED frees slot

ACTIVE $\rightarrow$ ARCHIVED frees slot

Also revoke distribution/publishing on disable/archive.

Limit reached messaging

“You’ve used 3/3 Active Agents. Disable or archive an active agent, or upgrade your plan.”

Sharing & Publishing Model

Before activation

PRIVATE (Disable Member and Role Tab)

After activation

PRIVATE - still private by default (Enable Member and Role Tab)

Sharing scopes (only meaningful when **ACTIVE**)

- **Enable Member and Role Tab**

Publishing channels (only when **ACTIVE**)

- Widget/embed code
- WhatsApp
- Messenger

UX Requirements

Agent list/card

Must show:

- Status badge: Draft/Test/Active/Disabled/Archived
- Owner
- If Active: “Licensed Active” indicator
- Quick actions by state: Test, Activate, Disable, Archive, Share, Publish

Share/Publish UI gating

If agent not Active:

- Share/Publish controls disabled with tooltip:
 - “Activate agent to enable distribution.”

License usage indicator (Admin + optionally all users)

- “Active Agents: 2 / 3”

Admin Requirements

Admin (or org owner) can:

- View entitlement: max_active_agents
- View usage: active_agents_count
- List active agents with:
 - owner, created_at, activated_at, last_run_at
- View license metadata:
 - deployment_id/install_id
 - expiry (if any)
- Replace/update license (on-prem) / plan changes (SaaS)

Edge Cases & Rules

- Clone ACTIVE agent → clone is DRAFT
- Attempt share/publish on non-ACTIVE → blocked
- Disable ACTIVE → immediately revokes access + frees slot
- Archive ACTIVE → frees slot, irreversible
- License downgrade (new max < current active):
 - do not auto-disable agents
 - block new activations
 - admin banner: “Reduce active agents to <= new limit.”

Use Cases

Use case A: Experimentation (free)

Owner builds agent, connects integrations, tests privately.

 allowed

 cannot share/publish

Use case B: Department deployment (licensed)

Owner activates agent then shares to a team.

 consumes 1 slot

 team can use agent

Use case C: External deployment (licensed)

Owner activates agent then deploys via widget / WhatsApp / Messenger.

 consumes 1 slot

 external access enabled

Use case D: Limit reached

Owner tries to activate agent #4 but limit is 3.

 blocked with clear guidance

User Stories

Story 1 — Private testing with integrations

As a builder

I want to test agents privately using integrations

So that I can validate workflows before deploying

Accept: integrations work; only owner sees outputs; no sharing allowed.

Story 2 — Activation consumes a slot

As an admin/owner

I want activating an agent to consume a license slot

So that deployed footprint matches entitlements

Accept: ACTIVE increments usage; logged.

Story 3 — Block activation at limit

As an admin/owner

I want activation blocked when quota is reached

So that we don't exceed license

Accept: state unchanged; error returned; UI message shows usage.

User Stories

Story 4 — Distribution requires Active

As a user

I want share/publish/embed/channel deployment only when Active

So that licensing aligns with multi-user access

Accept: non-ACTIVE attempts blocked.

Story 5 — Admin visibility

As an admin

I want dashboards and inventory reports

So that I can manage deployed agents

Accept: usage + list visible; export optional.

Technical Architecture Details

Problem constraints

- Deployment may be air-gapped
- No cloud callback allowed
- Must be self-contained and auditable

Components

1. Install/Deployment ID generator

- On install, generate stable deployment_id (UUID)
- Store in local config/DB/secret store

2. Signed License File

- Delivered by vendor to customer
- Contains:
 - customer_name
 - deployment_id (binding)
 - max_active_agents
 - optional valid_from / valid_until
 - signature
- Format:
 - JWT (signed) or JSON + detached signature

3. Local License Validator

- Verifies license signature using embedded public key
- Confirms deployment_id matches local installation
- Exposes entitlements to app services:
 - getMaxActiveAgents()
 - isValid()
 - getExpiry()

4. Activation Gatekeeper

- Implement inside API service that handles agent state transitions
- On activate, call validator + count active agents
- Enforce hard limit

5. Audit Logger

- Record events locally:
 - license upload
 - verify success/failure
 - activation/disable/archive
 - denial reasons

6. Admin UI

- Reads local license status + agent inventory
- Shows banners for expiry or invalid license

Flows

On-Prem Flow (end-to-end)

- Install generates deployment_id
- Vendor issues license bound to deployment_id
- Admin uploads license to atenxion
- Validator verifies signature + binding
- Activation checks entitlement and toggles state

Anti-tamper recommendations (practical)

- Asymmetric signing: vendor private key, product public key
- Deployment binding via UUID (not CPU/MAC)
- License stored in protected config, admin-only access

Expiry handling (recommended)

- On expiry: block new activations
- Optional grace for existing ACTIVE agents (configurable)
- Admin warnings (UI + logs)

Thank You!

